

What is Batch???

- You may be having a data set of huge size, say, a million brain scan images. So, if you load the whole data set into the memory, the training speed of the model will be very slow because you are using a lot of memory in your CPU which is very inefficient.
- You can break your data set into batches, that is, if you have a data set containing ten brain scan images, you can split your data set into two batches where each batch has five images,

Total images = 10

1 batch = 5 images

So, a total of two batches.

(Number of batches * Number of images in a single batch = Total number of data set) $\Rightarrow (2 * 5 = 10)$.

What is Batch???

- Let's get bigger, if you have a brain scan image data set containing 100000 images, we can convert it into 3125 batches where each batch has 32 images in it.

Total images = 100000

1 batch = 32 images

So, a total of 3125 batches, ($3125 * 32 = 100000$).

- So, instead of loading the whole 100000 images into memory which is way too expensive for the computer, you can load 32 images(1 batch) for 3125 times which requires way less memory as compared to loading the complete data set.

Epoch, Batch Size, and Iterations

- An **epoch** is when all the training data is used at once and is defined as the total number of iterations of all the training data in one cycle for training the machine learning model.

epoch

1

- **Batch Size** is the total number of training examples present in a single batch

Batch Size

2

- **Iterations** is the number of batches needed to complete one epoch

Iterations

3

We can divide the dataset of 2000 examples into batches of 500 then it will take 4 iterations to complete 1 epoch.

Solve the following problem

Solve the following classification problem with the perceptron rule. Apply each input vector in order, for as many repetitions as it takes to ensure that the problem is solved. Draw a graph of the problem only after you have found a solution.

$$\left\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\right\} \left\{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\right\} \left\{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\right\} \left\{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\right\}$$

Use the initial weights and bias:

$$\mathbf{b}(0) = 0, \mathbf{w}(0) = [0 \quad 0]$$

Solve the following problem

Solve the following classification problem with the perceptron rule. Apply each input vector in order, for as many repetitions as it takes to ensure that the problem is solved. Draw a graph of the problem only after you have found a solution.

$$\left\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\right\} \left\{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\right\} \left\{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\right\} \left\{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\right\}$$

Use the initial weights and bias:

$$\mathbf{b}(0) = 0, \mathbf{w}(0) = [0 \quad 0]$$

Solution

The desired output is either 1 or zero

So the suitable activation function is the binary step function (Hard limit function)

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the first epoch:

$$b(0) = 0, w(0) = [0 \ 0], \alpha = 1$$

For the first input vector:

$$n = w(0)P_1 + b(0)$$

$$n = [0 \ 0] \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0 = 0$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(0) = 1$$

$$e = t_1 - a = 0 - 1 = -1$$

$$w(1) = w(0) + \alpha e P_1^T$$

$$w(1) = [0 \ 0] + (1)(-1)[2 \ 2] = [-2 \ -2]$$

$$b(1) = b(0) + \alpha e$$

$$b(1) = 0 + (1)(-1) = -1$$

For the 2nd input vector:

$$n = w(1)P_2 + b(1)$$

$$n = [-2 \ -2] \begin{bmatrix} 1 \\ -2 \end{bmatrix} - 1 = 1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(1) = 1$$

$$e = t_2 - a = 1 - 1 = 0$$

$$w(2) = w(1) = [-2 \ -2]$$

$$b(2) = b(1) = -1$$

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the first epoch:

$$b(0) = 0, w(0) = [0 \quad 0], \alpha = 1$$

For the third input vector:

$$n = w(2)P_3 + b(2)$$

$$n = [-2 \quad -2] \begin{bmatrix} -2 \\ 2 \end{bmatrix} - 1 = -1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-1) = 0$$

$$e = t_3 - a = 0 - 0 = 0$$

$$w(3) = w(2) = [-2 \quad -2]$$

$$b(3) = b(2) = -1$$

For the fourth input vector:

$$n = w(3)P_4 + b(3)$$

$$n = [-2 \quad -2] \begin{bmatrix} -1 \\ 1 \end{bmatrix} - 1 = -1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-1) = 0$$

$$e = t_4 - a = 1 - 0 = 1$$

$$w(4) = w(3) + \alpha e P_4^T$$

$$w(4) = [-2 \quad -2] + (1)(1)[-1 \quad 1] = [-3 \quad -1]$$

$$b(4) = b(3) + \alpha e$$

$$b(4) = -1 + (1)(1) = 0$$

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the second epoch:

$$b(0) = 0, w(0) = [-3 \quad -1], \alpha = 1$$

For the first input vector:

$$n = w(0)P_1 + b(0)$$

$$n = [-3 \quad -1] \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0 = -8$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-8) = 0$$

$$e = t_1 - a = 0 - 0 = 0$$

$$w(1) = w(0) = [-3 \quad -1]$$

$$b(1) = b(0) = 0$$

For the 2nd input vector:

$$n = w(1)P_2 + b(1)$$

$$n = [-3 \quad -1] \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 0 = -1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-1) = 0$$

$$e = t_2 - a = 1 - 0 = 1$$

$$w(2) = w(1) + \alpha e P_2^T$$

$$w(2) = [-3 \quad -1] + (1)(1)[1 \quad -2] = [-2 \quad -3]$$

$$b(2) = b(1) + \alpha e$$

$$b(2) = 0 + (1)(1) = 1$$

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the second epoch:

$$b(0) = 0, w(0) = [-3 \quad -1], \alpha = 1$$

For the third input vector:

$$n = w(2)P_3 + b(2)$$

$$n = [-2 \quad -3] \begin{bmatrix} -2 \\ 2 \end{bmatrix} + 1 = -1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-1) = 0$$

$$e = t_3 - a = 0 - 0 = 0$$

$$w(3) = w(2) = [-2 \quad -3]$$

$$b(3) = b(2) = 1$$

For the forth input vector:

$$n = w(3)P_4 + b(3)$$

$$n = [-2 \quad -3] \begin{bmatrix} -1 \\ 1 \end{bmatrix} + 1 = 0$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(0) = 1$$

$$e = t_4 - a = 1 - 1 = 0$$

$$w(4) = w(3) = [-2 \quad -3]$$

$$b(4) = b(3) = 1$$

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the third epoch:

$$b(0) = 1, w(0) = [-2 \quad -3], \alpha = 1$$

For the first input vector:

$$n = w(0)P_1 + b(0)$$

$$n = [-2 \quad -3] \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 1 = -9$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-9) = 0$$

$$e = t_1 - a = 0 - 0 = 0$$

$$w(1) = w(0) = [-2 \quad -3]$$

$$b(1) = b(0) = 1$$

For the 2nd input vector:

$$n = w(1)P_2 + b(1)$$

$$n = [-2 \quad -3] \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 1 = 5$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(5) = 1$$

$$e = t_2 - a = 1 - 1 = 0$$

$$w(2) = w(1) = [-2 \quad -3]$$

$$b(2) = b(1) = 1$$

$$\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\} \{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\} \{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\} \{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\}$$

For the third epoch:

$$b(0) = 1, w(0) = [-2 \quad -3], \alpha = 1$$

For the third input vector:

$$n = w(2)P_3 + b(2)$$

$$n = [-2 \quad -3] \begin{bmatrix} -2 \\ 2 \end{bmatrix} + 1 = -1$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(-1) = 0$$

$$e = t_3 - a = 0 - 0 = 0$$

$$w(3) = w(2) = [-2 \quad -3]$$

$$b(3) = b(2) = 1$$

For the fourth input vector:

$$n = w(3)P_4 + b(3)$$

$$n = [-2 \quad -3] \begin{bmatrix} -1 \\ 1 \end{bmatrix} + 1 = 0$$

$$a = \text{hardlimit}(n) = \text{hardlimit}(0) = 1$$

$$e = t_4 - a = 1 - 1 = 0$$

$$w(4) = w(3) = [-2 \quad -3]$$

$$b(4) = b(3) = 1$$

The algorithm has converged the final solution is

$$w = [-2 \quad -3]$$

$$b = 1$$

Now we can graph the training data and the decision boundary of the solution. The decision boundary is given by

$$n = \mathbf{W}\mathbf{p} + b = w_{1,1}p_1 + w_{1,2}p_2 + b = -2p_1 - 3p_2 + 1 = 0.$$

To find the p_2 intercept of the decision boundary, set $p_1 = 0$:

$$p_2 = -\frac{b}{w_{1,2}} = -\frac{1}{-3} = \frac{1}{3} \quad \text{if } p_1 = 0$$

To find the p_1 intercept, set $p_2 = 0$:

$$p_1 = -\frac{b}{w_{1,1}} = -\frac{1}{-2} = \frac{1}{2} \quad \text{if } p_2 = 0$$

The resulting decision boundary is illustrated in Figure P4.6.

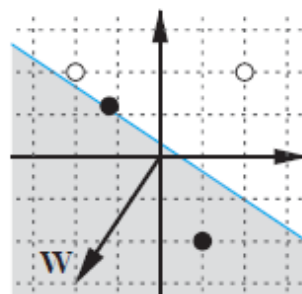


Figure P4.6 Decision Boundary for Problem P4.4

Note that the decision boundary falls across one of the training vectors. This is acceptable, given the problem definition, since the hard limit function returns 1 when given an input of 0, and the target for the vector in question is indeed 1.

Solve the following classification problem with the perceptron rule. Apply each input vector in order, for as many repetitions as it takes to ensure that the problem is solved. Draw a graph of the problem only after you have found a solution.

$$\left\{P_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0\right\} \left\{P_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1\right\} \left\{P_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0\right\} \left\{P_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1\right\}$$

Use the initial weights and bias:

$$\underline{b}(0) = 0, w(0) = [0 \quad 0]$$

**Report : solve the above problem with
learning rate $\alpha = 0.5$**

**Solve the following classification problem with the perceptron rule.
Apply each input vector in order, make only one epoch.**

$$\left\{ P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1 \right\} \left\{ P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1 \right\} \left\{ P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1 \right\}$$

Initial weight

$$b(0) = 1, w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.5$$

**Solve the following classification problem with the perceptron rule.
Apply each input vector in order, make only one epoch.**

$$\left\{ P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1 \right\} \left\{ P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1 \right\} \left\{ P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1 \right\}$$

Initial weight

$$b(0) = 1, w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.5$$

Solution

The desired output is either 1 or -1

So the suitable activation function is the sign function (symmetrical Hard limit function)

$$\left\{ P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1 \right\} \left\{ P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1 \right\} \left\{ P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1 \right\}$$

For the first epoch:

$$b(0) = 1, w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.5$$

For the first input vector:

$$n = w(0)P_1 + b(0)$$

$$n = [1 \quad -1 \quad 0 \quad 0.5] \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix} + 1 = 2.5$$

$$a = \text{hardlims}(n) = \text{hardlims}(2.5) = 1$$

$$e = t_1 - a = -1 - 1 = -2$$

$$w(1) = w(0) + \alpha e P_1^T$$

$$w(1) = [1 \quad -1 \quad 0 \quad 0.5] + (0.5)(-2)[1 \quad -2 \quad 0 \quad -1]$$

$$w(1) = [1 \quad -1 \quad 0 \quad 0.5] + [-1 \quad 2 \quad 0 \quad 1]$$

$$w(1) = [0 \quad 1 \quad 0 \quad 1.5]$$

$$b(1) = b(0) + \alpha e$$

$$b(1) = 1 + (0.5)(-2) = 0$$

$$\left\{ P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1 \right\} \left\{ P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1 \right\} \left\{ P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1 \right\}$$

For the first epoch:

$$b(0) = 1, w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.5$$

For the second input vector:

$$n = w(1)P_2 + b(1)$$

$$n = [0 \quad 1 \quad 0 \quad 1.5] \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix} + 0 = 0$$

$$a = \text{hardlims}(n) = \text{hardlims}(0) = 1$$

$$e = t_2 - a = -1 - 1 = -2$$

$$w(2) = w(1) + \alpha e P_2^T$$

$$w(2) = [0 \quad 1. \quad 0 \quad 1.5] + (0.5)(-2)[0 \quad 1.5 \quad -0.5 \quad -1]$$

$$w(2) = [1 \quad -1 \quad 0 \quad 0.5] + [0 \quad -1.5 \quad 0.5 \quad 1]$$

$$w(2) = [1 \quad -2.5 \quad 0.5 \quad 1.5]$$

$$b(2) = b(1) + \alpha e$$

$$b(2) = 0 + (0.5)(-2) = -1$$

$$\left\{ P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1 \right\} \left\{ P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1 \right\} \left\{ P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1 \right\}$$

For the first epoch:

$$b(0) = 1, w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.5$$

For the third input vector:

$$n = w(2)P_3 + b(2)$$

$$n = [1 \quad -2.5 \quad 0.5 \quad 1.5] \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix} - 1 = -5.75$$

$$a = \text{hardlims}(n) = \text{hardlims}(-5.75) = -1$$

$$e = t_3 - a = 1 - (-1) = 2$$

$$w(3) = w(2) + \alpha e P_3^T$$

$$w(3) = [1 \quad -2.5 \quad 0.5 \quad 1.5] + (0.5)(2)[-1 \quad 1 \quad 0.5 \quad -1]$$

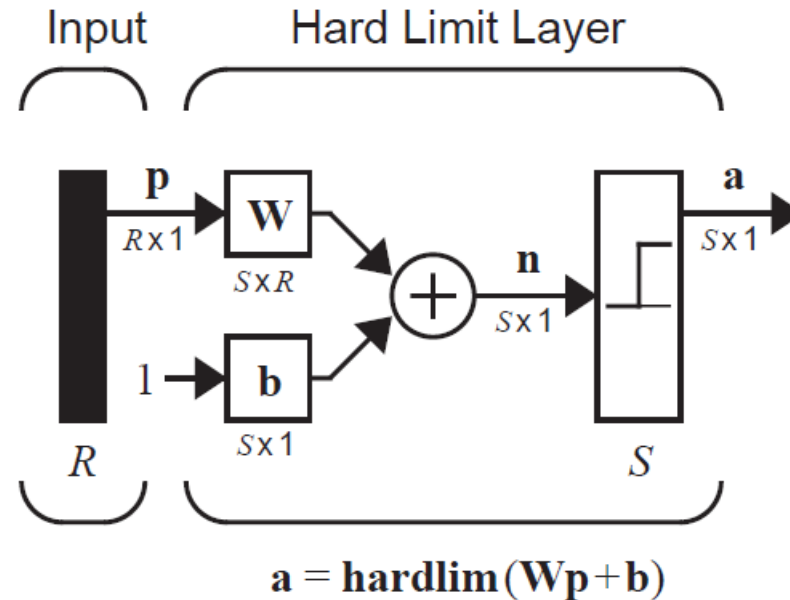
$$w(3) = [1 \quad -2.5 \quad 0.5 \quad 1.5] + [-1 \quad 1 \quad 0.5 \quad -1]$$

$$w(3) = [0 \quad -1.5 \quad 1 \quad 0.5]$$

$$b(3) = b(2) + \alpha e$$

$$b(3) = -1 + (0.5)(2) = 0$$

General Network Architectures



the S-neuron, R-input, one-layer network also can be drawn in abbreviated notation. The symbols below the variables tell you that for this layer, $\mathbf{p}$ is a vector of length R , $\mathbf{W}$ is an $S \times R$ matrix, and $\mathbf{b}$ and $\mathbf{a}$ are vectors of length S . As defined previously, the layer includes the weight matrix, the summation and multiplication operations, the bias vector $\mathbf{b}$, the transfer function boxes and the output vector.

Multiple-Neuron Perceptron

(more than 2 classes)

Solve the following problem

- 4- Consider again the four-class decision problem that we introduced in Problem P3. Train a perceptron network to solve this problem using the perceptron learning rule.

sol

If we use the same target vectors that we introduced in Problem P4.3, the training set will be:

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{t}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{t}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \mathbf{t}_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\} \\ \left\{ \mathbf{p}_4 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \mathbf{t}_4 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\} \left\{ \mathbf{p}_5 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \mathbf{t}_5 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\} \left\{ \mathbf{p}_6 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \mathbf{t}_6 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\} \\ \left\{ \mathbf{p}_7 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \mathbf{t}_7 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\} \left\{ \mathbf{p}_8 = \begin{bmatrix} -2 \\ -2 \end{bmatrix}, \mathbf{t}_8 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

Let's begin the algorithm with the following initial weights and biases:

$$\mathbf{W}(0) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{b}(0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

sol

To solve a problem with four classes of input vector we will need a perceptron with at least two neurons, since an S -neuron perceptron can categorize 2^S classes. The two-neuron perceptron is shown in Figure P4.2.

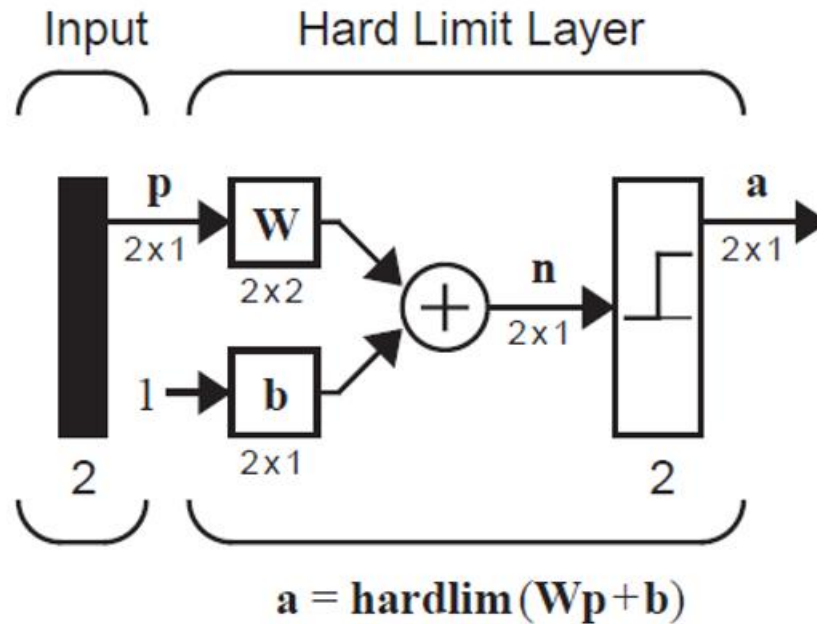


Figure P4.2 Two-Neuron Perceptron

The first iteration is

$$\mathbf{a} = \text{hardlim}(\mathbf{W}(0)\mathbf{p}_1 + \mathbf{b}(0)) = \text{hardlim}\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$\mathbf{e} = \mathbf{t}_1 - \mathbf{a} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix},$$

$$\mathbf{W}(1) = \mathbf{W}(0) + \mathbf{e}\mathbf{p}_1^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix},$$

$$\mathbf{b}(1) = \mathbf{b}(0) + \mathbf{e} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The second iteration is

$$\mathbf{a} = \text{hardlim}(\mathbf{W}(1)\mathbf{p}_2 + \mathbf{b}(1)) = \text{hardlim}\left(\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

$$\mathbf{e} = \mathbf{t}_2 - \mathbf{a} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

$$\mathbf{W}(2) = \mathbf{W}(1) + \mathbf{e}\mathbf{p}_2^T = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix},$$

$$\mathbf{b}(2) = \mathbf{b}(1) + \mathbf{e} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The third iteration is

$$\mathbf{a} = \text{hardlim} (\mathbf{W}(2)\mathbf{p}_3 + \mathbf{b}(2)) = \text{hardlim} \left(\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$\mathbf{e} = \mathbf{t}_3 - \mathbf{a} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix},$$

$$\mathbf{W}(3) = \mathbf{W}(2) + \mathbf{e}\mathbf{p}_3^T = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 & -1 \end{bmatrix} = \begin{bmatrix} -2 & 0 \\ 1 & -1 \end{bmatrix},$$

$$\mathbf{b}(3) = \mathbf{b}(2) + \mathbf{e} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Iterations four through eight produce no changes in the weights.

$$\mathbf{W}(8) = \mathbf{W}(7) = \mathbf{W}(6) = \mathbf{W}(5) = \mathbf{W}(4) = \mathbf{W}(3)$$

$$\mathbf{b}(8) = \mathbf{b}(7) = \mathbf{b}(6) = \mathbf{b}(5) = \mathbf{b}(4) = \mathbf{b}(3)$$

The ninth iteration produces

$$\mathbf{a} = \textit{hardlim} (\mathbf{W}(8)\mathbf{p}_1 + \mathbf{b}(8)) = \textit{hardlim} \left(\begin{bmatrix} -2 & 0 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

$$\mathbf{e} = \mathbf{t}_1 - \mathbf{a} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$

$$\mathbf{W}(9) = \mathbf{W}(8) + \mathbf{e}\mathbf{p}_1^T = \begin{bmatrix} -2 & 0 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix},$$

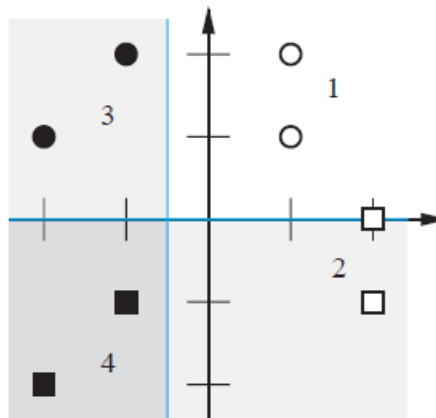
$$\mathbf{b}(9) = \mathbf{b}(8) + \mathbf{e} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Decision Boundary (1):

- $(-2)p_1 + (0)p_2 + (-1)b = y$
- Replacing y with 0, $-2p_1 - b = 0$
- Since bias, $b = 1$, so $-2p_1 - 1 = 0$
- $p_1 = -0.5$

Decision Boundary (2):

- $(0)p_1 + (-2)p_2 + (0)b = y$
- Replacing y with 0, $-2p_2 = 0$
- $p_2 = 0$



Performance Measurement

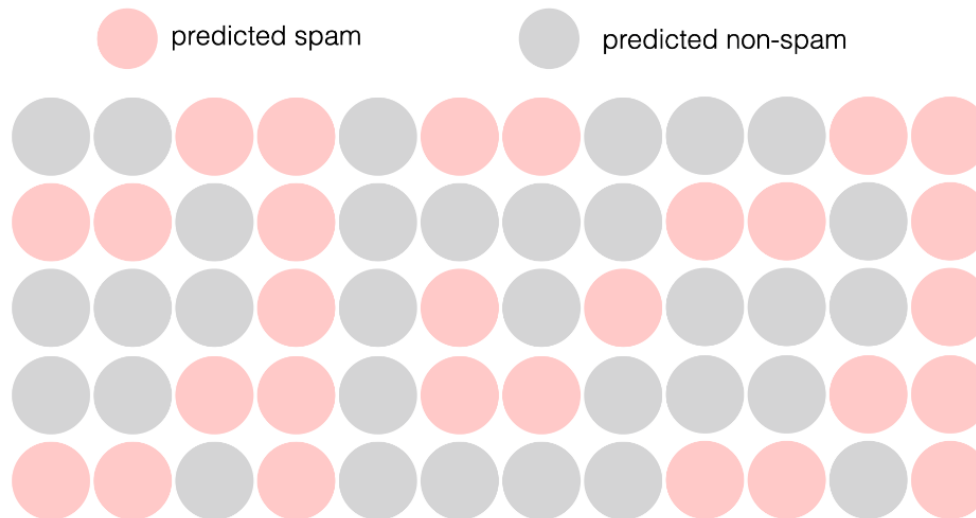
- **Accuracy** is a metric that measures how often a machine learning model correctly predicts the outcome. You can calculate accuracy by dividing the number of correct predictions by the total number of predictions

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

You can measure the accuracy on a scale of 0 to 1 or as a percentage. The higher the accuracy, the better. You can achieve a perfect accuracy of 1.0 when every prediction the model makes is correct.

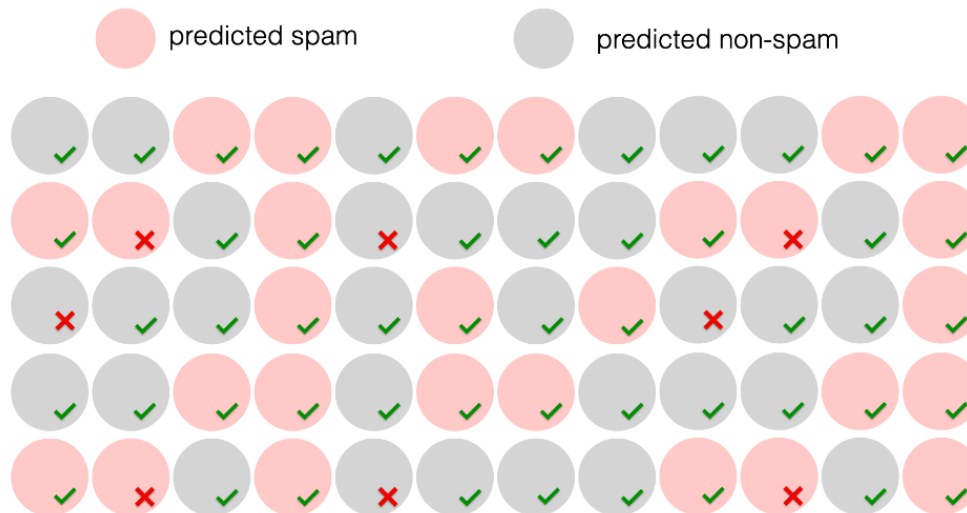
Accuracy example

- Let's say we have a machine learning model that performs spam detection.
- For every email in the dataset, it produced a prediction and assigned one of the two classes: "spam" or "not spam." Here is how we can visualize the **predicted** labels:



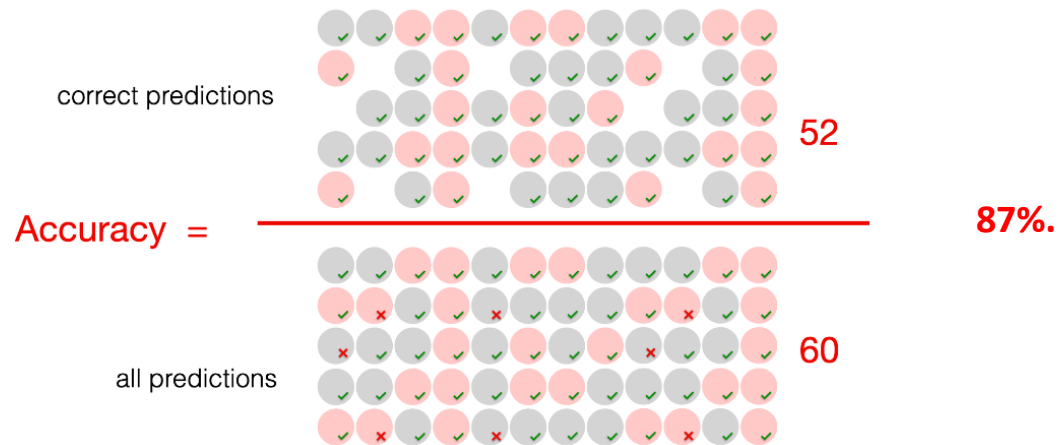
Accuracy example

- After we get the **actual** labels (learn which emails are spam and which are not), we can evaluate if the model predictions are correct.
- We can visually tag each correct and wrong prediction.



Accuracy example

- Now, you can simply count the number of times the model was right and divide it by the total number of predictions.
- In our case, 52 out of 60 predictions (labeled with a green “tick” sign) were correct. Meaning the **model accuracy is**

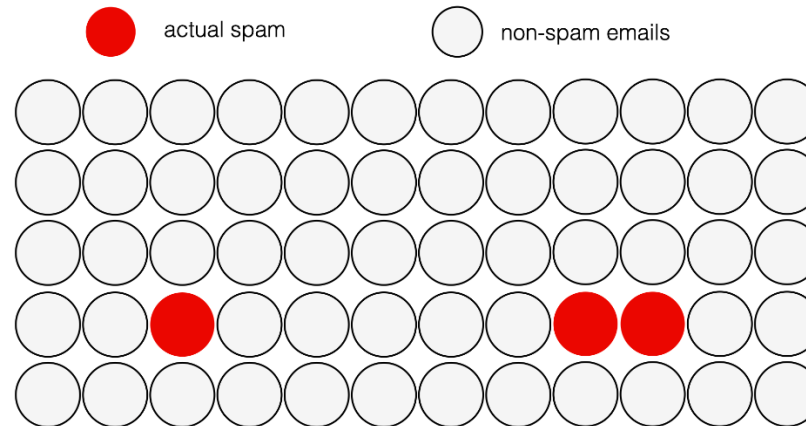


Accuracy paradox

- There is a downside to focusing on accuracy as a primary metric. The reason is that it treats all classes as equally important and looks at all correct predictions.
- In the example above, there are two classes, and the dataset is reasonably balanced. We have multiple cases of both spam and non-spam emails. In these cases, accuracy can be a suitable and helpful metric. For example, this might happen when you classify the objects (think “cat or dog” type of use cases) and have enough examples of each class.
- However, many real-world applications have a **high imbalance of classes**. These are the cases when one category has significantly more frequent occurrences than the other.

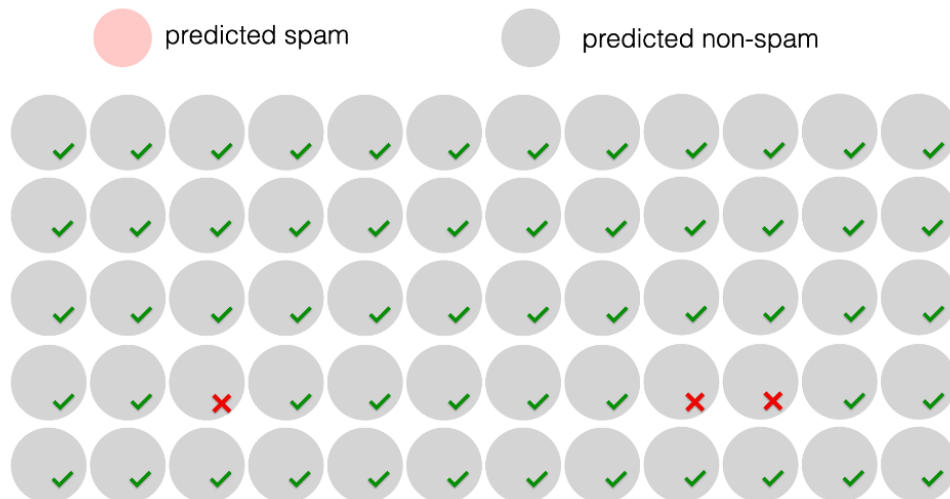
Accuracy paradox

- For example, this might happen when you are predicting payment fraud, equipment failures, users churn, or identifying illness on a set of X-ray images. In scenarios like this, you are typically interested in predicting the **events that rarely occur**. They might only happen in 1-10% of cases or even less frequently.
- Let's illustrate the idea. Imagine that the actual balance of spam and non-spam emails looks like this. Out of 60 emails, only 3 (or 5%) are truly spam.



Accuracy paradox

- It is easy to “game” the accuracy metric when making predictions for a dataset like this. To do that, you simply need to predict that nothing will happen and **label every email as non-spam**. The model predicting the majority (non-spam) class all the time will mostly be right, leading to very high accuracy.



- In this specific example, **the accuracy is 95%**: yes, **the model missed every spam email**, but it was still right in 57 cases out of 60.
- However, this accuracy is now **meaningless**. The model does not serve the primary goal and does not help identify the target event.

Pros and cons

Let's sum up the accuracy metric!

- **Pros:**

- Accuracy is a helpful metric when you deal with **balanced classes** and care about the overall model “correctness” and not the ability to predict a specific class.
- Accuracy is **easy to explain** and calculate.

- **Cons:**

- If you have **imbalanced classes**, accuracy is less useful since it gives equal weight to the model's ability to predict all categories.